SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	G.Vinay Kumar Reddy	**Reg. No.:**	192472093
Section / Batch:	CSE AI	**Date:**	29/08/2026

Code of Conduct

I G.vinay kumar reddy) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Gvinay Kumar Reddy

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."
Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking
Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.
Apply your algorithm to:
User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."
Your algorithm should maintain the following slots:
Departure City
Destination City
Travel Date
Number of Passengers
Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

1. Reference Resolution Using Multiple Constraints

Introduction

Reference resolution identifies a pronoun or referring expression and determines which previously mentioned entity it refers to. Multiple constraints such as **gender, number, recency, and semantic compatibility** can be combined to select the most appropriate antecedent.

Algorithm

Algorithm: Reference Resolution

Input: Discourse D

1. Divide D into sentences.
2. Identify all named entities and noun phrases.
3. Identify referring expressions such as he, she, it, them, her, etc.
4. For each referring expression:
    a. Generate previously mentioned entities as candidates.
    b. Check gender agreement.
    c. Check number agreement.
    d. Give higher score to recent candidates.
    e. Check semantic compatibility.
5. Assign a score to every valid candidate.
6. Select the candidate with the highest score.
7. Replace the referring expression with the selected antecedent.
8. Output the resolved discourse.

End

Application to the Given Discourse

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Step 1: Entities and Referring Expressions

Entities:

- Meena – female person
- Kavya – female person
- Laptop – object
- Project – object/activity

Referring expressions:

- she
- one
- her
- She
- her
- it

Step 2–4: Possible Antecedents and Constraints

Referring Expression	Possible Antecedents	Selected Antecedent	Reason
she	Meena, Kavya	Kavya*	Female and semantically likely to need a laptop for her project
one	laptop	Laptop	"one" refers to an object that is needed
her	Meena, Kavya	Kavya*	Refers to the person associated with the project
She	Meena, Kavya	Kavya*	Recent female participant and likely person receiving the laptop
her	Meena, Kavya	Meena*	"thanked her" identifies the other participant
it	laptop	Laptop	"using it" requires an object

*The first sentence contains genuine ambiguity because **Meena** and **Kavya** are both female and singular. Pure gender/number agreement cannot uniquely resolve every pronoun.

Step 5: Resolved Discourse

Meena gave Kavya a laptop because Kavya needed a laptop for Kavya's project. Kavya thanked Meena and started using the laptop.

Avoiding Incorrect Resolution

The algorithm should not depend on only one constraint. It can assign scores to candidates:

Score =

Gender agreement

+ Number agreement

+ Recency

+ Grammatical compatibility

+ Semantic compatibility

+ Discourse coherence

For example, both **Meena** and **Kavya** satisfy female and singular agreement. Therefore, the algorithm uses **semantic role, grammatical structure, recency, and discourse context** to distinguish them.

However, when all constraints remain equally plausible, the system should mark the reference as **ambiguous** rather than making an unjustified decision.

---

2. Algorithm for Entity Chains and Discourse Coherence

Introduction

An entity chain connects different expressions that refer to the same entity throughout a discourse. Strong entity chains generally indicate that the text is coherent and maintains its topic.

Algorithm

Algorithm: Entity Chain Detection

Input: Discourse D

1. Divide D into sentences.
2. Extract nouns, noun phrases, named entities, and pronouns.
3. Compare each expression with previously identified entities.
4. Use lexical similarity and semantic compatibility to identify references.
5. Link expressions referring to the same entity.
6. Create an entity chain for each entity.
7. Calculate entity continuity:

   Continuity = Number of sentence transitions
   containing a continuing entity
   / Total sentence transitions

8. Determine coherence from entity continuity and semantic consistency.
9. Output entity chains and coherence result.

End

Application

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Step 1: Entities

- Anjali
- laptop
- processor
- Python
- computer
- machine learning application

Step 2–3: Entity Chains

Person chain:

Anjali → She → Anjali

Laptop chain:

laptop → The laptop → it → the computer

Processor chain:

processor

Software chain:

Python

Application chain:

machine learning application

Entity Continuity

There are four sentence transitions:

Transition	Continuing Entity
S1 → S2	laptop
S2 → S3	laptop
S3 → S4	Anjali + laptop
Overall	Strong continuity

An approximate continuity measure is:

$$Continuity = \frac{\text{transitions containing continuing entities}}{\text{total transitions}}$$

Since the main entities continue across the sentences, the discourse has **high entity continuity**.

Coherence Decision

The discourse is coherent

The text consistently discusses **Anjali using a laptop for programming and machine learning**.

Effect of Breaking an Entity Chain

Suppose the final sentence were:

"Later, Anjali used the **car** to develop a machine learning application."

The laptop chain would be broken. The reader would have difficulty understanding how the previous discussion of the laptop connects to the new object. Therefore, breaking important entity chains can reduce **continuity, clarity, and discourse coherence**.

---

3. Algorithm for Identifying Discourse Relations

Introduction

Discourse relations describe the logical relationship between sentences. Common relations include **Cause–Effect, Sequence, Problem–Solution, and Elaboration**.

Algorithm

Algorithm: Discourse Relation Identification

Input: Discourse D

1. Divide D into individual sentences/discourse units.
2. Identify discourse markers such as:
   because, therefore, as a result,
   after that, however, then, etc.
3. Analyze important events and actions.
4. Determine the relationship between consecutive units.
5. Assign the most suitable discourse relation.
6. Connect the relations to construct a discourse structure.

7. Check whether the complete structure is logically coherent.

8. Output the discourse structure.

End

Application

Discourse Units

- **D1:** The server crashed unexpectedly.
- **D2:** As a result, users could not access the application.
- **D3:** The technical team restarted the server.
- **D4:** After that, the system became available again.

Discourse Markers

- **As a result** → Cause–Effect
- **After that** → Sequence

Relation Analysis

Units	Relation	Explanation
D1 → D2	Cause–Effect	Server crash caused users to lose access
D2 → D3	Problem–Solution	Access problem led to the technical team restarting the server
D3 → D4	Sequence	System availability followed the restart

Discourse Structure

D1: Server crashed

↓

CAUSE–EFFECT

↓

D2: Users lost access

↓

PROBLEM–SOLUTION

↓

D3: Team restarted server

↓

SEQUENCE

↓

D4: System became available

Coherence

The discourse is logically coherent because the events follow a clear progression:

Problem → Effect → Solution → Result

The explicit markers **"As a result"** and **"After that"** make the relationships easier for an NLP system to identify.

---

4. Algorithm for Dialogue Act Classification

Introduction

Dialogue Act Recognition determines the communicative purpose of an utterance. It helps a conversational agent understand whether the user is greeting, requesting information, asking a question, providing information, or making an offer.

Algorithm

Algorithm: Dialogue Act Classification

Input: Conversation C

1. Preprocess each utterance.
2. Tokenize and normalize the text.
3. Extract linguistic features:
    - Question words
    - Imperatives
    - Greeting words
    - Request expressions
    - Information statements
    - Offer expressions
4. Determine the communicative intention.
5. Assign one dialogue act.
6. Store the dialogue acts in conversation order.
7. Generate the dialogue-act sequence.

End

Application

Conversation Analysis

Speaker	Utterance	Dialogue Act	Reason
User	"Hello, I need help with my assignment."	Greeting + Request	"Hello" is a greeting and "need help" is a request
Agent	"Sure, what problem are you facing?"	Question	Requests information from the user
User	"I do not understand machine learning."	Inform	Provides information about the user's problem
Agent	"I can explain the basic concepts to you."	Offer	Offers assistance

Dialogue-Act Sequence

Greeting + Request

↓

Question

↓

Inform

↓

Offer

Importance

Dialogue-act recognition helps an agent select an appropriate response.

For example:

User → Request

Agent → Provide information

User → Question

Agent → Answer question

User → Inform

Agent → Acknowledge or offer assistance

Therefore, the conversational agent can maintain the flow of the conversation instead of generating unrelated responses.

---

5. Algorithm for Dialogue State Tracking

Introduction

Dialogue State Tracking (DST) maintains important information throughout a multi-turn conversation. In a flight-booking system, the chatbot stores information such as **departure city, destination city, travel date, and number of passengers**.

Algorithm

Algorithm: Dialogue State Tracking

Input: User and Agent conversation

1. Initialize all dialogue slots as EMPTY.
2. Read each user utterance.
3. Identify entities and values in the utterance.
4. Match each value with the appropriate slot.
5. Update the dialogue state.
6. Check which required slots are still EMPTY.
7. Select the next missing slot.
8. Generate a question requesting that information.
9. Continue until all required information is collected.

10. Confirm the completed booking details.

End

Initial Dialogue State

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

Turn 1

User:

"I want to book a flight."

The system recognizes the intent:

Flight Booking

No slot values are provided.

State

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

The next required information is the **Destination City**.

Agent:

"Where would you like to travel?"

Turn 2

User:

"I want to go to Delhi."

The system extracts:

Destination City = Delhi

Updated State

Slot	Value
Departure City	Unknown
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

The next missing slot is **Departure City**.

Agent:

"From which city?"

Turn 3

User:

"From Chennai."

The system extracts:

Departure City = Chennai

Final State After Given Conversation

Slot	Value
Departure City	Chennai
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

The chatbot should next ask:

"What is your travel date?"

After receiving the date, it should ask for the number of passengers if that information is still missing.

Importance of Dialogue State Tracking

Dialogue state tracking allows the chatbot to remember information across multiple turns. Without it, the agent might ask for **Delhi** or **Chennai** again even after the user has already provided them.

Thus, DST provides:

- **Context awareness**
- **Entity consistency**
- **Reduced repetition**
- **Correct follow-up questions**
- **Coherent multi-turn interaction**

Overall Conclusion

Reference resolution, entity-chain analysis, discourse-relation detection, dialogue-act recognition, and dialogue-state tracking are important components of conversational NLP. They allow systems to understand **who or what is being discussed, how sentences are related, what the user intends, and what information has already been provided**, resulting in more coherent and context-aware conversations

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	3	3	3	3	3	15	75
2	3	3	3	3	3	15	
3	3	3	3	3	3	15	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 29/08/226	Date: _____	Date: _____